Srijan Vyom(creation space)/ "आकाश का निर्माण" या "रचनात्मक आसमान" Transformation

Phase 1: Updated Architectural Decisions & Tools

We are evolving the codebase shown in your screenshots (image_3.png) from a static site into a dynamic, full-stack application.

Core Platform Stack

- **Database (DB): MongoDB Atlas (Great for storing user profiles, project metadata, and connection states).**
- **File Storage (DAM): Cloudinary (Yes, it allows storing PDFs as "raw" files. Continue using it for screenshots and project images).**
    - ***Note on Code Uploads:* In a 5-day sprint, do not build your own version control system. Instead, allow users to submit links to their existing GitHub/GitLab repositories. The SV dashboard will "embed" that project metadata.**
- **Authentication: Clerk (Quickest path to secure, user/guest, social login).**

AI & Advanced Feature Stack

- **AI Models: Gemini Pro (Text/Multimodal) (Free tier available, good Node.js support).**
- **AI Framework: LangChain for JavaScript (Integrates Gemini, handles RAG (Retrieval-Augmented Generation) for PDF summarization).**
- **Chat DB: Store chat history as a sub-document structure in MongoDB connected to the specific project/user.**

Infrastructure

- **Frontend Deployment: Vercel (Connect your GitHub repo).**
- **Backend Deployment: Use Vercel Functions (Serverless) if keeping it simple, or a simple Node/Express server on Render/Railway if the AI operations time out.**

Phase2: PRE-WORK: Setup (Optional Day 0)

- Finalize DB schemas (Users, Projects, Conversations).
- Initialize Clerk/Atlas/Cloudinary projects.
- Get Gemini API keys.

DAY 1: Platform Foundation & Authorization

- **Goal:** A locked backend, established DB connection, and secure login states.
- **Back-end:** Set up a Node/Express server. Connect MongoDB Atlas.
- **Auth:** Integrate Clerk into both frontend and backend.
    - Create roles (Admin, Signed-in User, Guest). Guests can view; Users can modify.
- **Front-end:** Update the main navigation (image_2.png) to include Clerk <UserButton> / <SignInButton>.

DAY 2: Core Platform UI & Upload

- **Goal:** A working upload system and project data persistence.
- **Back-end:** Create API routes for project submission (Title, Desc, Stack, GitHub URL, Cloudinary asset references).
- **Cloudinary Integration:** Implement frontend upload widget logic to get screenshot/PDF URLs.
- **Front-end:**
    - Create the "Submit Project" form page (visible only to users).
    - Update the ProjectGrid.jsx to pull dynamic data from your MongoDB API rather than static arrays.

DAY 3: Search & Detailed Project View

- **Goal:** Find projects and see their full details.
- **Back-end:** Create search endpoints using MongoDB Atlas Search (fast, flexible search by user/project/tag).
- **Front-end:**
    - Build the central search bar UI.
    - Finalize the detailed "focused view" page (image_1.png) that loads specific project data from your API.

DAY 4: LangChain AI: PDF & Code Summarizer

- **Goal:** Gemini summarization is live.

- **Implementation:**
    - Create a dedicated LangChain agent in your backend.
    - For PDFs: Use LangChain to extract text from a PDF URL (from Cloudinary) -> Send to Gemini -> Get summary.
    - For Code: Since we use GitHub links, this is tougher in 5 days. For the MVP, prioritize summarizing text descriptions or *extracted* code snippets rather than full repo analysis.
- **Front-end:** Add the "Generate Summary" button to the detailed project page.

DAY 5: Simplified Social & Notification

- **Goal:** Basic user-to-user visibility and admin broadcast.
- **Features:**
    - **Handshake (Social):** Simplified connection logic in the DB. A "connected user" can view download links.
    - **Notification:** A simple "Inbox" in the user profile showing admin broadcasts.
- **Front-end:** Update the user profile section to show connected users.

DAY 6: AI Chat Storage & Admin Dashboard

- **Goal:** A persistent chat experience and operational dashboard.
- **AI Chat:**
    - Update the summarizer logic to store the result in your Conversations collection in MongoDB.
    - Connect the existing community chatbot UI (from your original widgets) to make actual API calls to Gemini.
- **Admin Dashboard:** Create a basic React dashboard to view all projects/users and moderation controls.
- **Final Review:** End-to-end testing, cleanup, and initial Vercel deployment.